

Technologies Web

Master Informatique

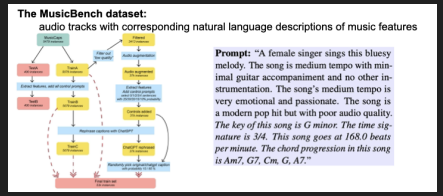
2026/2027

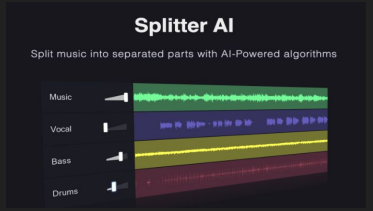
Michel Buffa

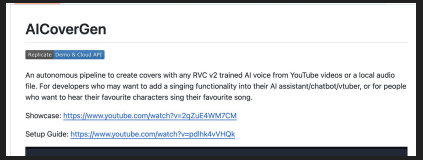
michel.buffa@univ-cotedazur.fr

LINK OF THIS COURSE : https://tinyurl.com/939xcxet

Université Côte d'Azur / Laboratoire I3S / INRIA, WIMMICS group







UNIVERSITÉ CÔTE D'AZUR | ÉCOLE UNIVERSITAIRE DE RECHERCHE SYSTÈMES NUMÉRIQUES POUR L'HUMAIN | FRANCE 2030 Initiative d'Excellence | i3S sophia antipolis | Inria | cnrs | wimmics

Who am I?

- Professor / researcher at Université Côte d'Azur (UCA), France
  - Member of the WIMMICS research group common to INRIA and I3S lab from CNRS
  - W3C Advisory Committee Representative for my university
  - Member of the W3C WebAudio Working Group since 2014
  - Main Research topic: Web Audio
  - michel.buffa@univ-cotedazur.fr

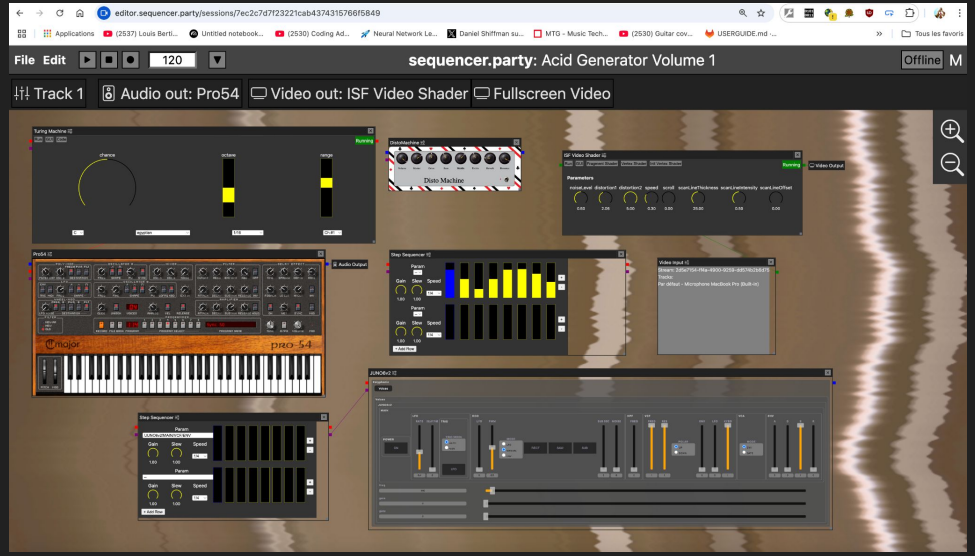


- Involved in the Web Audio Modules (WAM) plug-in standard, aka "VSTs for the Web"
- Teach "Using AI for Music Creation" at MSc Music Scoring for Visual Media and Sound Design (Cannes) and at Master Création contemporaine et nouvelles technologies option Informatique Musicale (Université Jean Monnet, Saint-Etienne)

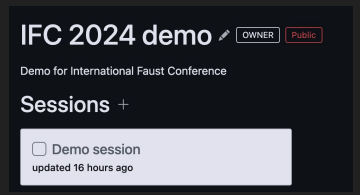


Buffa, M., Ren, S., Campbell, O., Burns, T., Yi, S., Kleimola, J. and Larkin, O., 2022, April. Web Audio Modules 2.0: An Open Web Audio Plugin Standard. In *Companion Proceedings of the Web Conference 2022* (pp. 364-369).

Real-time collaborative improvisation, 2D and 3D/Immersive !



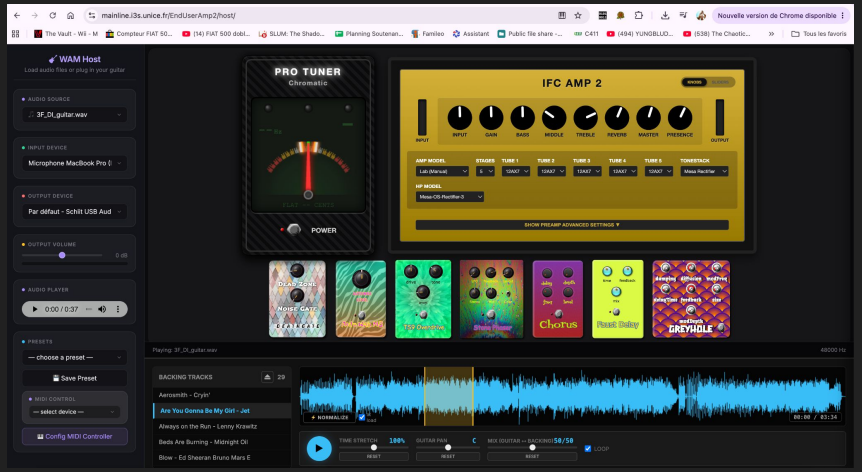




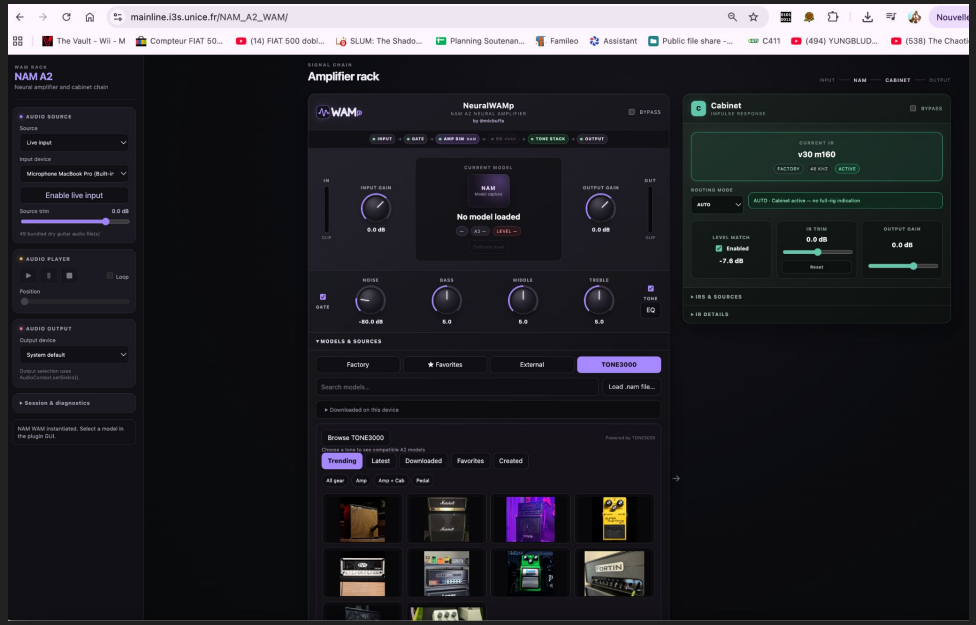
Buffa, M., Girard, D., Demont, S., Escobar, Q. and Hofr, A., 2024, November. Faust Plugins in (Sometimes Unexpected) Web-Based Hosts. In *International Faust Conference 2024*.

Buffa, M., Girard, D. and Hofr, A., 2024, September. Using Web Audio Modules for Immersive Audio Collaboration in the Musical Metaverse. In *2024 IEEE 5th International Symposium on the Internet of Sounds (IS2)* (pp. 1-10). IEEE.

Tube guitar amp simulations / audio effects



Modélisation de circuits électroniques Web Components + format de plugins réutilisables WAM + Cross compilation en Web Assembly depuis le langage FAUST.



Utilisation de profils NAM A2 (réseaux de neurones) qui “capturent” le comportement des amplificateurs à lampes

Tools we’re going to use

Some don’t need installation:

- Online IDEs : jsbin.com, codepen.io, codesandbox.io etc.

Some need to be installed

- Visual Studio code, NodeJS, Git and eventually WebStorm
- Github copilot (free with GitHub Education) / Gemini / Codex / Claude / Antigravity / Cursor

Also: not so up to date list of tools for web development and some recommendations about how to configure your Mac for web dev.

Software development is being automatized

A revolution is on its way. I stopped writing code six months ago..

… But I've been programming for decades, my expertise is super high on my domains of applications.

How to teach Web Technologies today ?

AI assistants are super human… (see this: how GTA San Andreas has been reverse engineered to rebuild the complete source code from binaries)

What should YOU learn ?

How can I evaluate your learning?

Web Technologies? What is it exactly?

Front-End

- **Real web standards**:
  - Langages: HTML, CSS, JavaScript, WebAssembly
    - Cross compilation…
  - Protocols : http/s, websockets, webRTC…
  - W3C web browsers' APIs: more than 50 APIs!
    - QuerySelector, DOM, Pointer Events, Drag and Drop, Forms, Tables, Multimedia (audio/video/webcam/micro streamés), Canvas, Geolocation, Motion, IndexedDB, Fetch, WebNN, Web Codecs, MediaDevices, WebGL, WebGPU, etc.
  - Web Components: Templates, Shadow Dom, Custom Elements
- Libraries and Frameworks (open source, industry…) :
  - Stencil, Lit, Angular, React, VueJS, Tailwind CSS, etc.
- Build tools:
  - Vite, WebPack, Parcel, Rollup, etc.

Web Technologies? What is it exactly?

Back-End

- **HTTP Servers**
  - Can serve HTML pages (with CSS/JavaScript and assets:images, icons, sounds, files etc.)
  - Often provide **Web Services / API / REST endpoints**
  - Asynchronous communication
- **WebRTC / WebSockets servers**
  - **For real-time, synchronous communication**
- These server use web standards, but are NOT standardized: made by the open source Community or private companies
- Large variety of servers
  - Java EE (Tomcat, Spark Java, Spring Boot, etc.)
  - Python (Flask, Django, etc.)
  - JavaScript/TypeScript: **Node JS / Express** et frameworks associés (très nombreux)

Framework	Points forts	Cas d'utilisation
Express.js	Minimaliste, large communauté	API REST simples.
Sails.js	MVC, WebSockets intégrés	Applications temps réel ou API complexes.
Koa.js	Middleware léger, modernité	Serveurs flexibles et personnalisables.
NestJS	Architecture modulaire	Applications d'entreprise robustes.
Hapi.js	Validation intégrée, riche en fonctionnalités	API structurées pour applications complexes.
AdonisJS	Inspiré de Laravel, prêt à l'emploi	Applications CRUD et backend performants.
Fastify	Ultra performant, support JSON Schema	Serveurs à haute performance.
Meteor.js	Framework full-stack	Applications en temps réel avec gestion client/serveur.
LoopBack	Génération automatique d'API	Backend basé sur des bases de données existantes.

Web Technologies? What is it exactly?

Databases: traditional SQL and NoSQL ones:

- PostgresSQL, MySQL, Oracle, SQL Server, etc.
- MongoDB, Cassandra, CouchDB, Neo4J, etc.

Real-time multi-user collaborative databases:

- FireBase/FireCloud (real-time, websocket-based)
- Redis (clusters)

Related tools (indexer, cache, etc.):

- Elastic Search, etc.

données	Type	Points forts	Cas d'utilisation
MongoDB	Document	JSON flexible, évolutif	Applications dynamiques.
Cassandra	Colonnes	Haute disponibilité, haute tolérance	Big Data, temps réel.
Redis	Clé-valeur	Ultra rapide, stockage en mémoire	Caching, messages en temps réel.
Elasticsearch	Recherche & analyse	Recherche rapide, analytique puissante	Moteurs de recherche.
Couchbase	Document/Clé-valeur	Requêtes SQL-like	Applications nécessitant des performances élevées.
DynamoDB	Clé-valeur	Gestion automatique (AWS)	Cloud, microservices.
Firebase	Clé-valeur	Synchronisation temps réel	Applications mobiles, jeux multi-utilisateurs.
Neo4j	Graphes	Analyse des relations complexes	Réseaux sociaux, chaînes logistiques.
HBase	Colonnes	Big Data sur Hadoop	Journaux massifs.

Web Technologies? What is it exactly?

Hosting / Cloud / Virtualization

- Many possibilities
  - Simple project, without a back-end : GitHub pages, CloudFlare, etc.
  - Project with back-end:
    - **What to deploy?**
      - Files, Docker image, etc.?
      - How to organize yourself for local development testing and easy deployment (CI/CD: Continuous Integration / Continuous Deployment, part of DEVOPS)
    - **Hosting**
      - render.com, Google Cloud Platform, Microsoft Azure, Amazon, Heroku, OVH
      - Dedicated Server (proxy, security)

What are we going to see in this course?

This is a brand new course

Front-End

- Basics
  - MOOCs at w3cx.org : HTML Coding Essentials and best practices, HTML5 Apps and Games, JavaScript Introduction,

- Asynchronous programming with JavaScript/TypeScript and use related Web browser APIs : Fetch API, JavaScript promises, Async / Await, Table API, Forms API, etc.
- Other APIs: WebAudio, WebMidi, MediaDevices, DOM Api, persistence APIs etc.
- Web Components: principles, reusing WC, creating pure WC

- WebComponents through **Frameworks (Angular, React, NextJS…)**

This is a brand new course

Back-End

- We will use Node JS / Express / MongoDB (with Mongoose, sort of ORM for MongoDB)
  - Web Services, several npm modules for NodeJS
  - Authentication through JSON Web Tokens (JWT)
  - Maybe Single Sign On techniques (authenticate with Google/GitHub for example)

- Hosting on GitHub Pages and Cloud (render.com, mongodb.com)

This is a brand new course

Use of AI assistants

- I will try to share some best practices… but you need to be up to date and do technological watch regularly
- Best use of AI assistants
- New ways to evaluate your work… See guidelines in the TPs.

First exercices: run the TP exercise skeleton

You will need NodeJS!

Get it at: https://nodejs.org/

Get the stable version

Test: run node –version in a terminal

DO NOT USE POWERSHELL!

Get the skeleton for the exercises

```
git clone
```
https://github.com/micbuffa/TechnosWeb_M1_INFO_2026_2027

And read CONSEIL_POUR_UTILISER_ASSISTANT_AI.md, README.md and SUJET_ETUDIANT_TP1.md

Follow Teacher’s instructions.

Second exercises:
Try to do TP1 and start TP2
Take time to do carefully what is proposed (i.e look into the code, produce report etc.)

Third course

Angular

Let’s look at the detailed Angular Course…

Important concepts:

- Web Components
- Using modules
- Signals / bindings (how can we link view and model ?) - view = HTML, model = TypeScript variables
- Services (i.e proxy with the back-end / web services). In our TP app, where is the service ? How can we check with the debugger what is happening?
- Router : associate routes (URLs) with the component that should be displayed.
- Security (authentifications / authorizations)

Let’s look at TP2

Mission 2 / TP2 :

- Pagination, it is already partially implemented. Where ? How ?
- How are “Prec” and “Suiv” buttons become disabled when on first / last page ? How current page / number of pages are updated?
- Use component Paginator from Angular Material. Try to add it using prompt and check what has changed in the source files. Where is the documentation that can help explain how this component works ?
- What is the Mongoose plugin aggregate-paginate-v2 and what does it do? It is worth using it in the project?

Let’s look at TP2

Mission 3 / TP2 :

- Improve upload and audio file playback:
  - Frontend Error handling (existing file, bad format etc.)
  - Explain why frontend validation improves the user experience but never replaces backend validation.
- Add or complete only the missing UI elements during submission:
  - display a loading state; progress bar…
  - disable the upload button while an upload is active and prevent duplicate submissions;
  - display server errors;
  - display a success message;
  - Display the tracks as responsive and accessible cards. Each card can show the title, original filename, format, size, date added, and a playback action.
  - Delete track

Let’s look at TP2

Mission 3 / TP2 :

- Extract id3 tags from audio files, show metadata (artist, song name)
- Option for finding and displaying the song / album cover image
- Better looking audio player
    - BTW: how is the audio player implemented in HTML / Typescript ?

Fourth course

Starting from the end of Lab 3

Be able to You are “more free” to orient your project…

Think about what you could do with a prototype that has:

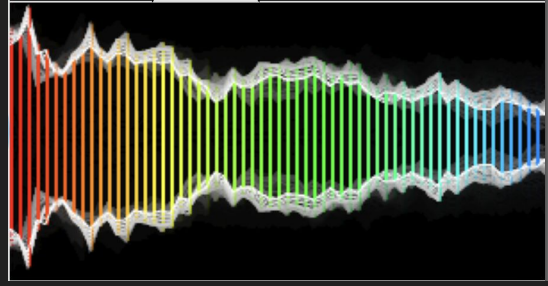
- User management
- Known how to upload files and other kind of data

You could:

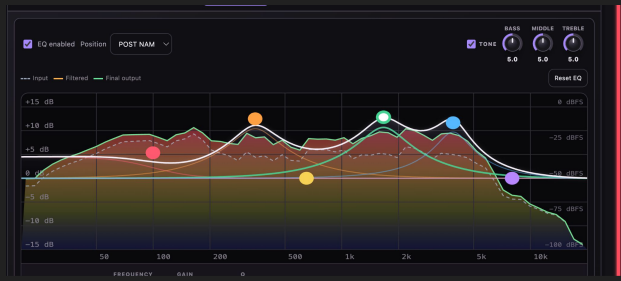
- Make a small social network for sharing files + multimedia capabilities
- Make a solo user application but with private and public files
- Make a music oriented application (my speciality)

Let me give some suggestions…

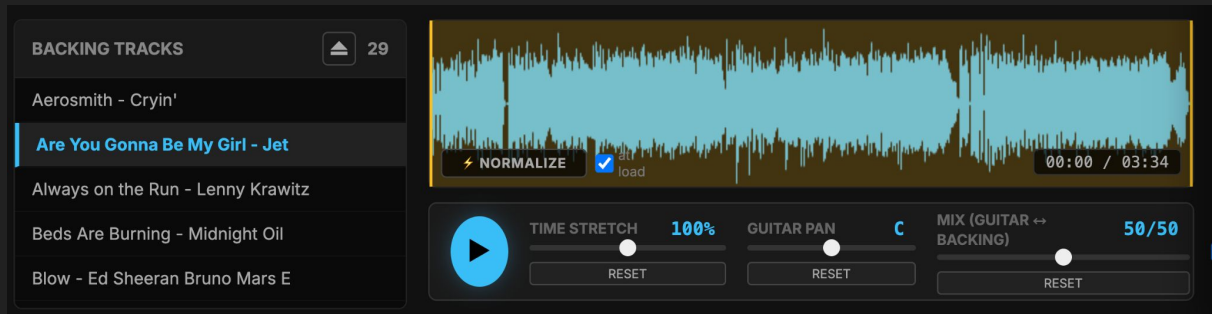
Improve audio player



Customize UI and exploits [the WebAudio API]():



- Display waveform
- Stereo vu-meters for volume left/right
- Display dancing frequencies
- Use custom GUI widgets (rotating knobs, sliders, switches)
- Additional controls from UI: panning L/R, graphic EQ for adjusting sound
- Add audio effects (reverb, distorsion, delay, filter (cutoff / resonance)
- See my WebAudio course on https://www.edx.org/learn/html5/the-world-wide-web-consortium-w3c-html5-apps-and-games (free), chapter 1.5.
- Add graphic background effects (use of webgl/webgpu shaders that can react to the music, check for example the butterchurn project).



Public / private files

Be able to share files with other users:

- A file can be public and searched by everyone
- Display somewhere “newly added files”
- A file can also be private
  - And you can share with other “friends”
  - So: you need a sort of social network with invite/add/remove friends/see friends activity etc.
  - You can stop sharing files
- You need a search

Before implementing, discuss with your AI assistant about 1) the features you would like 2) The proposed UIs 3) the implementation 4) cut the implementation in different tasks/missions/steps and start by implementing the first one, ask to add unit tests for all new features etc. Ask for a report in REPORT.md file. etc.